MODULE 15 · DOMAIN 2 - SECURITY, COMPLIANCE, AND GOVERNANCE

Application Authentication and Authorisation

Sign users and services in with tokens, decide what they may do in one policy engine, and give pods their own identity.

DOMAIN 2 of 5 EXAM WEIGHT 25% MODULE 15 of 52

admin@sthithapragna.com

Authentication answers **who you are**.
Authorisation answers **what you may do**.
Put each where it belongs.

Cognito user pool or identity pool?

USER POOL

Who are you?

- Sign-up, sign-in, MFA, account recovery
- Social, SAML and OIDC federation
- Issues ID and access tokens
- Groups become claims in tokens

IDENTITY POOL

What may you touch in AWS?

- Exchanges a token for AWS credentials
- Maps users to IAM roles
- Policy variables scope each user
- Optional unauthenticated guest role

THE TIEBREAKER

If users call AWS services directly, such as S3 from a mobile app, add an identity pool.

OAuth 2.0 flows worth knowing

FLOW	WHO USES IT	NOTE
Authorization code with PKCE	Web and mobile apps signing in users	The default for user sign-in
Client credentials	Service to service, no user	Cognito charges per token issued
Refresh token	Apps renewing access quietly	Revocable; keeps sessions short
Scopes	Any token	Limit what the token allows

ON THE EXAM

No human in the loop means client credentials. A person signing in means authorization code with PKCE.

Cognito user pool feature plans

PLAN	ADDS
Lite	Core sign-up and sign-in, social and enterprise federation
Essentials	Default: managed login, passwordless, access token customisation
Plus	Threat protection: adaptive authentication, compromised credentials

ON THE EXAM

Risk-based MFA and compromised-password detection are Plus features. Passkeys and managed login are Essentials.

Authorising requests at API Gateway

AUTHORIZER	VALIDATES	CHOOSE IT WHEN
Cognito authorizer	User pool tokens and scopes	REST API, users in Cognito
JWT authorizer	Any OIDC or OAuth issuer	HTTP API, standard tokens
IAM authorization	SigV4-signed AWS principals	Callers are AWS roles
Lambda authorizer	Anything your code checks	Custom logic or Verified Permissions

ON THE EXAM

Fine-grained rules beyond group membership are usually a Lambda authorizer calling Verified Permissions.

Rules in code, or in Verified Permissions?

AUTHORISATION IN CODE

Scattered logic

- Rules repeated in every service
- Changes need a deployment each time
- Hard for auditors to review
- Drifts between services

AMAZON VERIFIED PERMISSIONS

One policy engine

- Cedar policies in a policy store
- Services ask IsAuthorized at runtime
- Role and attribute based rules
- Policies reviewable and versioned centrally

THE TIEBREAKER

The same permission rules duplicated across services, or auditors needing to read them, means Verified Permissions.

Verified Permissions in practice

- **Policy store** holds Cedar policies and a schema of principals, actions and resources.
- **Identity source:** a Cognito user pool or OIDC provider, so tokens become principals.
- **API-linked policy stores** generate a Lambda authorizer for API Gateway.
- **Rules on attributes:** region, role, amount, tenant, combined with groups.
- **Authorizer decisions are cached** for 120 seconds by default; revocation lags.

ON THE EXAM

Verified Permissions decides application actions. IAM still decides access to AWS resources.

Each pod should have its own AWS permissions, not borrow the node's.

EKS Pod Identity or IRSA?

EKS POD IDENTITY

AWS's recommended default

- No OIDC provider per cluster
- Roles trust one service principal
- Same role reused across clusters
- Session tags for ABAC

IAM ROLES FOR SERVICE ACCOUNTS

Still needed in places

- OIDC provider for each cluster
- Trust policy names each cluster
- Required on Fargate and Windows
- Also EKS Anywhere, self-managed Kubernetes

THE TIEBREAKER

Pod Identity whenever supported. Fargate pods, Windows nodes and EKS Anywhere still need IRSA.

Reading the scenario

WHEN THE QUESTION SAYS	IT IS TESTING
Mobile users upload straight to S3	User pool plus identity pool
Partner backends, no human user	Client credentials grant with scopes
Risk-based MFA, compromised passwords	Cognito Plus plan
Same permission rules in many services	Verified Permissions with Cedar
Cluster upgrades break role trust	EKS Pod Identity
Pods on Fargate need AWS access	IRSA, not Pod Identity

ON THE EXAM

Authentication questions turn on who the caller is. Authorisation questions turn on where the rules live.

> Meridian Freight's driver app lets drivers sign in and upload proof-of-delivery photos straight to Amazon S3 using the AWS SDK, to avoid routing large files through the backend.
>
> Each driver must only be able to write to their own prefix in the bucket.

What should the solutions architect implement?

A. A Cognito user pool for sign-in, and an identity pool whose role policy uses the identity ID as a prefix variable

B. A Cognito user pool with an API Gateway Cognito authorizer

C. An IAM user for each driver with an access key in the app

D. An identity pool with unauthenticated access only

◆ PAUSE AND DECIDE

What should the solutions architect implement?

A User pool plus identity pool

Temporary AWS credentials per driver, confined to their own prefix by a policy variable.

B API Gateway authorizer

Protects an API, but uploads would route through the backend, which the design avoids.

C IAM user per driver

Long-lived keys inside an app on thousands of phones.

D Unauthenticated only

No sign-in, so drivers cannot be told apart or confined.

THE SIGNAL

Users calling an AWS service directly is the identity pool's purpose. The policy variable does the confinement.

> Three logistics partners' backend systems will call Meridian Freight's rate API automatically. No people are involved in these calls.
>
> Each partner must receive tokens limited to the rate endpoints, and tokens must expire within an hour.

What should the solutions architect implement?

A. API keys in an API Gateway usage plan for each partner

B. An IAM user with access keys for each partner

C. Managed login pages where partners sign in each day

D. A Cognito app client per partner using the client credentials grant, with custom scopes for the rate endpoints

◆ PAUSE AND DECIDE

What should the solutions architect implement?

A **API keys**

Identify callers for quotas, but never expire and are not an authorisation mechanism.

B **IAM users**

Long-lived keys, and partners become principals in your account.

C **Managed login**

For people. These are machines.

D **Client credentials with scopes**

Short-lived tokens per partner, limited by scope to the rate endpoints.

THE SIGNAL

Machine to machine, expiring tokens, limited scope: the client credentials grant.

> Meridian Freight's customer portal uses a Cognito user pool on the Essentials plan. It is facing credential-stuffing attacks using passwords leaked from other sites.
>
> The company wants extra verification only when a sign-in looks risky, and to block known compromised passwords.

What should the solutions architect do?

A. Add an AWS WAF rate-based rule to the sign-in endpoint

B. Turn on passwordless sign-in with passkeys

C. Move the user pool to the Plus plan and enable threat protection with adaptive authentication

D. A pre-authentication Lambda trigger that counts failed attempts

◆ PAUSE AND DECIDE

What should the solutions architect do?

A **WAF rate limit**

Slows bulk attempts, but cannot judge risk per sign-in or spot leaked passwords.

B **Passkeys**

Strong where adopted, but existing password users remain exposed.

C **Plus plan threat protection**

Risk-based extra verification and compromised-credential checks, built in.

D **Pre-auth trigger**

Custom counting code, with no risk scoring or leaked-password data.

THE SIGNAL

Adaptive authentication and compromised-credential detection are Plus plan features, not Essentials.

> Rules deciding which Meridian Freight dispatchers may approve which shipments depend on the dispatcher's region, role and the shipment's value. The rules are coded separately in six microservices and have drifted apart.
>
> Auditors want to review the rules in one place, and changes should not require redeploying services.

What should the solutions architect recommend?

A. A shared authorisation library imported by each service

B. Amazon Verified Permissions, with the rules as Cedar policies and services calling it for decisions

C. IAM policies attached to each dispatcher's role

D. Cognito groups for each combination of region and role

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A Shared library

Reduces drift, but every rule change means redeploying six services.

B Verified Permissions

One reviewable policy store; services ask at runtime; no redeploy for rule changes.

C IAM policies

IAM governs AWS API calls, not application objects such as a shipment's value.

D Cognito groups

Cannot express a rule on the shipment's value, and multiplies groups.

THE SIGNAL

Application rules on attributes, reviewed centrally, changed without redeploys. That is a policy engine.

> Meridian Freight runs 30 EKS clusters on Linux EC2 nodes and replaces clusters during upgrades. Each replacement requires editing the trust policy of every IAM role used by IRSA.
>
> The platform team wants pod-level permissions without per-cluster trust changes.

What should the solutions architect recommend?

A. EKS Pod Identity, with roles trusting the EKS pods service principal and associations per service account

B. Keep IRSA and script the trust policy updates

C. Grant all required permissions to the node instance role

D. Store access keys in Kubernetes secrets

◆ PAUSE AND DECIDE

What should the solutions architect recommend?

A EKS Pod Identity

Trust names a service principal, not a cluster, so new clusters need no edits.

B Scripted IRSA updates

Automates the problem instead of removing it.

C Node instance role

Every pod on the node gets every permission. Not pod-level.

D Keys in secrets

Long-lived credentials inside the cluster.

THE SIGNAL

IRSA trust names each cluster's OIDC provider. Pod Identity's trust names none, which is the fix.

> A Meridian Freight team runs a new EKS workload entirely on AWS Fargate. Its pods need to read from one DynamoDB table and write to one SQS queue.
>
> Each pod must have only those permissions, without long-lived credentials.

How should the pods obtain AWS credentials?

A EKS Pod Identity associations for the service account

B The Fargate pod execution role, with DynamoDB and SQS permissions added

C IAM roles for service accounts, using the cluster's OIDC provider

D Access keys mounted from a Kubernetes secret

◆ PAUSE AND DECIDE

How should the pods obtain AWS credentials?

A **EKS Pod Identity**

Not supported for pods running on Fargate.

B **Pod execution role**

Used by Fargate to pull images and write logs, not by the application.

C **IRSA**

Supported on Fargate, scoped per service account, temporary credentials.

D **Keys in a secret**

Long-lived credentials, which the scenario rules out.

THE SIGNAL

Fargate is the deciding word. Pod Identity is the default elsewhere, but not there.

What to carry out of this module

- **User pool proves who; identity pool grants AWS access** when users call AWS directly.
- **Client credentials** for machines; **authorization code with PKCE** for people.
- **Plus plan** for adaptive authentication and compromised-password detection.
- **Verified Permissions** for application rules, written in Cedar, reviewed centrally.
- **Pod Identity by default; IRSA** on Fargate, Windows and EKS Anywhere.

> ON THE EXAM
>
> IAM governs AWS APIs. Verified Permissions governs your application's own actions.